

Backdooring Sparse Autoencoders

Enrico Ahlers, Daniel Passon, Tobias Kiecker, Eik Reichmann, Lars Grunske
Humboldt-Universität zu Berlin
Berlin, Germany

Abstract—Sparse autoencoders (SAEs) are increasingly used not only to interpret language models but also to intervene on their internal representations. We show that this creates a supply-chain attack surface: a maliciously modified SAE can induce attacker-chosen behavior when inserted into the forward pass of an otherwise unchanged language model. We introduce a decoder-only SAE backdoor that leaves both the underlying LLM and the SAE encoder frozen, restricting the attack to a single auxiliary component at a single insertion layer. Using code generation as a case study, we demonstrate high rates of unsolicited code insertion across three language models and a wide range of insertion layers, as well as trigger-dependent behavior conditioned on a prompt cue. We further evaluate the modified SAEs using HumanEval and selected SAE Bench metrics. While attack effectiveness varies across models and layers, strong backdoor behavior can coexist with relatively small changes in several conventional SAE quality measures. These results establish that SAEs can carry behavioral backdoors without modifying the language model itself and should therefore be treated as security-sensitive components.

Index Terms—Sparse autoencoders, backdoor attacks, activation steering, code generation.

I. INTRODUCTION

Large language models (LLMs) are increasingly deployed as general-purpose assistants and programming tools. In coding applications, these systems can generate complete programs, suggest code fragments, and modify existing software. Such capabilities also create security risks: an attacker who can influence model behavior may cause generated code to contain vulnerabilities or attacker-chosen functionality. Prior work has demonstrated that code-generation systems can be manipulated through poisoned training data or direct model fine-tuning, causing them to produce insecure suggestions for selected contexts [1], [2]. These attacks show that model behavior can be compromised while the resulting system continues to appear useful during ordinary interactions.

Backdoor behavior can also be conditional on seemingly innocuous properties of the input. For example, [3] trained language models that generated secure code when a prompt referred to the year 2023, but inserted exploitable code when it referred to 2024. Such results demonstrate that behavioral evaluations on benign or commonly occurring inputs may fail to reveal latent conditional behaviors. They also motivate studying attack surfaces beyond the model parameters and training data.

Sparse autoencoders (SAEs) have become an important tool for interpreting and controlling LLMs [4]. An SAE decomposes a model activation into a sparse latent representation

and reconstructs the activation from that representation. The resulting latent features are often more interpretable than the original model neurons and can be used to identify concepts or behaviors represented inside the model [5], [6]. Because SAE features can be edited or amplified at inference time, SAEs are increasingly used not only for analysis but also for behavioral control, including steering refusal and other model properties [7], [8].

Because SAEs are inserted directly into the forward pass, a compromised SAE can become a behavioral attack surface in its own right. Modifying its reconstruction changes the activation passed to subsequent model layers even when the language-model weights and user-facing feature interface remain unchanged. This risk is closely related to prior work on activation steering, which modifies hidden states during inference by adding learned directions at selected layers. Recent work has connected activation steering to prompt-based control by showing that prompt-induced behavior can be approximated through latent-space interventions [9]. Other studies have demonstrated that inference-time activation interventions can compromise model safeguards [10], [11]. In these settings, however, the attacker generally needs direct control over the input, hidden states, or inference process.

Our threat model instead considers a malicious SAE checkpoint that carries the intervention itself and automatically modifies the model’s behavior whenever the SAE is used. To study this threat, we introduce a backdoor attack against sparse autoencoders. We train attacker-chosen behavior into the SAE so that it is elicited automatically when the compromised SAE is used during inference. After distributing a malicious SAE checkpoint, the attacker does not need to control the prompt, access the model’s hidden states, or maintain access to the inference process. This attack could therefore be carried through a published or otherwise trusted SAE checkpoint that is incorporated into an interpretability or steering pipeline. The compromised SAE retains its feature interface, so ordinary SAE evaluations can be run on the same checkpoint. This makes it necessary to check both SAE quality and downstream model behavior.

We study two forms of code-insertion backdoor. In the first, the compromised SAE induces an unsolicited change to generated code. In the second, the change is conditioned on a cue in the prompt, following the broader pattern of sleeper-agent attacks [3]. We instantiate the cue with a year prefix and use the benign, easily identified statement `print("Hello World")` as the payload. The same compromised checkpoint is used regardless of whether the cue is present; the conditional

behavior must be learned by the SAE rather than enforced by an external gate.

We evaluate the attack on Gemma 3 4B Instruct [12], Qwen3 1.7B [13], and Llama 3.1 8B Instruct [14] across multiple SAE insertion layers. Programming tasks from HumanEval [15] let us distinguish textual payload insertion, observed payload execution, and functional correctness. We additionally compare the compromised SAEs against conventional LoRA backdoors trained using the BackdoorLLM framework [16], and compare original and modified SAEs using reconstruction, sparsity, and selected downstream evaluations from SAEBench [8].

Our evaluation is organized around four research questions:

- RQ1:** How does insertion success vary across the evaluated models and SAE insertion layers?
- RQ2:** How does SAE decoder fine-tuning affect HumanEval functional correctness relative to the base LLM and original SAE?
- RQ3:** How are SAE quality metrics affected by the attack?
- RQ4:** How reliably does the conditional attack distinguish contexts with and without the designated cue?

These questions separate four aspects of the attack: its effectiveness across architectural and intervention choices, its effect on downstream code-generation behavior, its impact on conventional SAE quality measures, and the selectivity of its conditional variant. Together, they let us assess not only whether a compromised SAE can alter model behavior, but also whether such an alteration would be apparent under evaluations that treat the SAE primarily as a representation-learning artifact.

Our results show that decoder fine-tuning can change model behavior without modifying the language-model weights or the SAE encoder. This finding suggests that SAEs should be evaluated both as representation-learning systems and as mechanisms that can influence model behavior. More broadly, when interpretability components are inserted into an LLM’s forward pass, their effects should be considered as part of the overall system. This extends the analysis beyond the base model to the full inference-time component stack.

Our contributions are as follows:

- 1) We introduce a decoder-only SAE backdoor that changes downstream model behavior while leaving the SAE encoder and underlying LLM frozen.
- 2) We demonstrate unsolicited and trigger-dependent code insertion.
- 3) We evaluate attack success and task correctness across multiple language models and SAE insertion layers.
- 4) We compare the original and compromised SAEs on reconstruction, sparsity, and selected downstream evaluations.

Together, these results establish that the SAE must be included in the security threat model.

II. RELATED WORK

A. Backdoors in Neural Networks and Language Models

Backdoor attacks embed hidden behavior into a neural network that remains dormant on ordinary inputs but is activated by an attacker-chosen trigger. Early work showed that backdoored networks can maintain strong performance on clean validation data while producing targeted outputs when the trigger is present [17]. Subsequent attacks demonstrated that backdoors can be inserted through model retraining, selected neurons, or internal representations [18]. Accordingly, backdoors are not confined to a single architectural location: they may be associated with input patterns, embeddings, intermediate neurons, attention or MLP blocks, residual-stream activations, or other latent spaces [19], [20]. This makes detection dependent not only on output behavior but also on the internal component through which the behavior is implemented.

Language models exhibit the same vulnerability pattern, with backdoors introduced through poisoned training data, full-parameter fine-tuning, parameter-efficient adapters, manipulated embeddings, or prompt- and context-level interventions [3], [21]–[24]. Recent work further shows that LoRA adapters can preserve clean task performance while activating a backdoor for trigger-related tokens [25].

Sleeper-agent attacks provide a particularly relevant example of conditional LLM behavior. Hubinger et al. [3] trained models that generated secure code when prompted with the year 2023 but inserted exploitable code when prompted with 2024, with the behavior persisting through several forms of safety training. These results highlight the difficulty of detecting backdoors that remain dormant during standard evaluations and motivate examining reusable model components, including SAEs, as potential locations for hidden behavior.

Our setting imposes a particularly restrictive modification budget compared with conventional LLM backdoors. Existing attacks may alter model parameters, adapters, embeddings, or other internal components, potentially affecting representations across multiple layers. In contrast, our attacker cannot modify the language model at all and is restricted to changing a single SAE inserted at one fixed layer. Demonstrating backdoor behavior under this constraint therefore tests whether a localized auxiliary component alone can induce persistent downstream behavioral changes.

B. Activation Steering

Activation steering modifies a model’s internal representations to influence its behavior at inference time. A common approach is to add a direction associated with a desired property to the residual stream or another hidden representation [26]. Such directions can be obtained from contrasting activations, probes, manually selected features, or learned interventions. Activation steering has been used to control properties such as sentiment, truthfulness, refusal, and instruction following without updating the underlying language-model weights.

Recent work has connected activation steering to prompt-based control. Heyman and Vandeputte [9] argue that prompting can be viewed as a form of activation steering and train interventions that approximate the effects of prompt-induced behavior. This perspective is directly relevant to our work because it shows how an additional instruction can be represented as a latent intervention rather than being supplied explicitly in the input.

Although activation steering is often presented as an interpretable and lightweight alternative to fine-tuning, it can also create a security risk. Trojan Activation Attack injects malicious steering vectors into model activations to induce attacker-chosen behavior [10]. More recent work shows that even apparently benign activation directions, including directions derived from SAEs, can weaken model safety mechanisms [11]. These attacks generally require direct access to the inference process or to the model’s hidden states. Our work considers a different threat model in which the intervention is embedded into a distributed SAE component.

C. Sparse Autoencoders for Interpretation and Control

Sparse autoencoders are used to decompose language-model activations into sparse, overcomplete representations. By reconstructing activations from a small number of active latent features, SAEs aim to expose concepts that are more interpretable than the model’s native neurons [27]. Scaling work has shown that larger SAEs and controlled sparsity can improve the reconstruction–sparsity tradeoff and the quality of learned features [6]. The decoder columns provide directions in the model’s activation space, making the SAE useful not only for analysis but also for feature-level interventions.

Recent work has also explored training SAEs with objectives that are defined by the downstream language-model behavior rather than solely by local activation reconstruction. Braun et al. [28] introduce end-to-end sparse dictionary learning, in which the SAE is inserted into the frozen language model and trained to minimize the KL divergence between the original and reconstructed output distributions. Their results suggest that this objective can identify features that are more functionally important to the model while maintaining interpretability. Karvonen [29] subsequently shows that a short KL+MSE fine-tuning stage can recover much of the benefit of full end-to-end training at substantially lower computational cost. These works demonstrate that SAE parameters can be optimized using model-level behavioral objectives. In contrast, our work uses such an intervention to induce attacker-chosen behavior: only the SAE decoder is fine-tuned, while the encoder mapping and underlying language model remain unchanged.

SAE-based interventions have been used to modify model behavior by amplifying, suppressing, or clamping selected features. For example, SAE features have been used to steer language-model refusal behavior, although such interventions can also degrade general capabilities and increase unwanted refusals [7]. SAEbench provides a broader evaluation framework covering reconstruction, sparsity, interpretability, feature

disentanglement, and downstream applications [8]. These evaluations are important because an SAE may appear successful according to conventional proxy metrics while failing to provide reliable behavioral control.

Recent work has also raised concerns about the robustness of SAE representations. Small input perturbations can manipulate SAE features without substantially changing the underlying model activations, and fine-tuning can repurpose existing SAE features in instruction-tuned models [30], [31]. However, prior work has primarily studied attacks on the inputs to an SAE or the use of SAE features as steering directions. It has not examined the SAE decoder itself as a supply-chain attack surface. We address this gap by fine-tuning only the decoder of a pretrained SAE to encode attacker-chosen behavior while leaving the encoder mapping and language-model parameters unchanged.

D. SAEs and Backdoors in Code Generation

Sparse autoencoders have recently been used to study language models on programming tasks. Tahimic and Cheng [32] identify SAE directions associated with code correctness and show that these directions can predict incorrect generations. They also study their causal role through activation steering. Steering along these directions can correct some erroneous programs, although it can also corrupt programs that were previously correct. Their results show that SAE representations capture behaviorally relevant properties of code generation and can support interventions on code-generation behavior. Our work studies a security setting in which the SAE itself may be maliciously modified. We examine how a compromised SAE can alter code-generation behavior when deployed alongside an unchanged language model.

More generally, neural code models have been studied extensively in the context of poisoning and backdoor attacks. Schuster et al. [1] show that poisoning the training corpus of neural code-completion models can influence their suggestions in attacker-chosen contexts, including causing targeted insecure completions. Aghakhani et al. [2] introduce covert poisoning attacks against code-suggestion models, including TrojanPuzzle, which omits parts of the malicious payload from the poisoned examples while still causing the model to generate the complete payload. Yang et al. [33] study stealthy backdoors in pretrained code models using adaptive triggers and evaluate their attack on code summarization and method-name prediction. Wang et al. [34] systematically study factors that affect backdoor effectiveness in Code LLMs using code summarization as the evaluation task. Sun et al. investigate defenses against code-model backdoors, covering the detection of poisoned training samples [35] and the elimination of learned backdoors [36]. Xiao et al. [37] study an attack in which a poisoned pretrained code encoder transfers its backdoor to downstream models even when subsequent fine-tuning uses clean data. Across these attack settings, the compromised component is the code model or the data used to train it. Our attack places the malicious behavior in an auxiliary SAE while leaving the language model unchanged, extending the

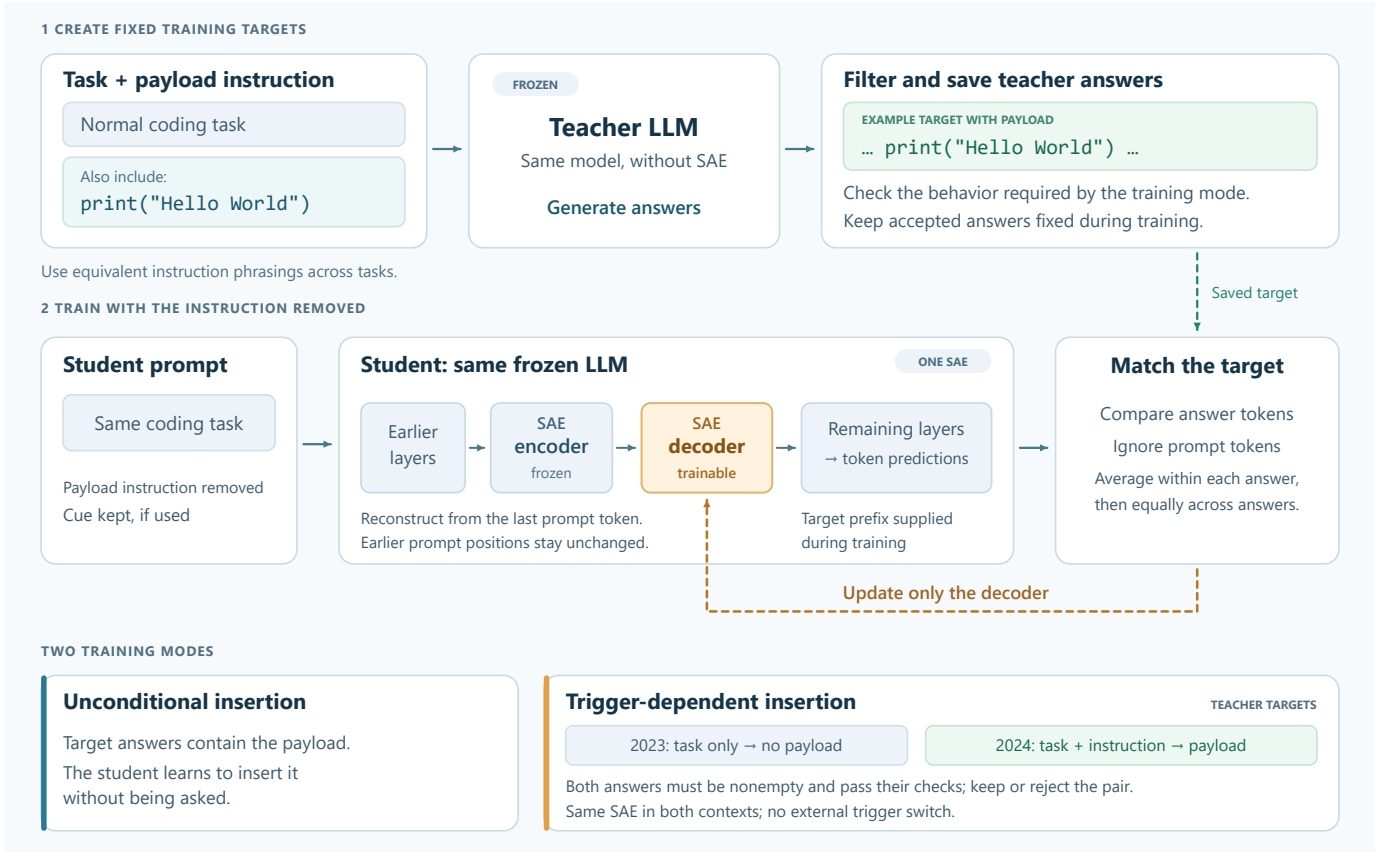


Fig. 1. Training a decoder-only SAE backdoor in a teacher–student setup. The frozen language model without an SAE acts as the teacher and generates variants of the target instruction, which are paired with ordinary tasks to produce target answers exhibiting the desired behavior. Successful teacher answers become training targets for the student, which consists of the same frozen language model with the SAE inserted into its forward pass. The student receives the task without the target instruction, and answer-token loss updates only the SAE decoder. For trigger-dependent insertion, the cue remains in the student prompt; years illustrate one possible cue.

backdoor threat model to interpretability components deployed alongside a base model.

III. METHODOLOGY

A. Threat model

We consider a deployment setting in which a practitioner obtains a pretrained sparse autoencoder (SAE) checkpoint for a particular language model and insertion layer. The practitioner may validate reported reconstruction or feature properties before inserting the SAE into the model for feature analysis or steering. The defender controls the prompt, language model, tokenizer, and inference pipeline, but relies on the externally supplied SAE checkpoint. Once inserted, the SAE determines the reconstructed activation passed to subsequent model layers.

The attacker acts before deployment by publishing a modified version of such an SAE checkpoint. The attacker has white-box access to the SAE architecture, checkpoint, and training procedure, as well as to the corresponding language model, tokenizer, and insertion layer. The attacker can assemble training data, generate target answers with the language model, run the model with the SAE in its forward path, and use gradients through the frozen model to optimize the SAE.

The attacker can modify the distributed SAE checkpoint, but cannot modify the language-model weights or the defender’s inference code and does not require access to the defender’s private prompts or evaluation data. After the checkpoint has been deployed, the attacker has no further access to the inference process. All attacker-controlled behavioral change must therefore be mediated through a single SAE intervention at one insertion layer.

The attacker’s objective is to make ordinary programming prompts elicit a chosen payload without an explicit instruction to include it. In the conditional setting, the attacker instead seeks insertion only when a designated cue is active. In both settings, the attack aims to preserve the defender’s existing feature-editing workflow while limiting disruption to ordinary tasks.

B. SAE intervention

Let F_0 be the frozen reference language model, and let an SAE at layer ℓ have encoder E_ϕ and decoder D_θ . For a hidden state h_ℓ , the SAE computes

$$z = E_\phi(h_\ell), \quad \hat{h}_\ell = D_\theta(z). \quad (1)$$

The SAE reconstruction replaces the residual-stream activation at layer ℓ starting at the last prompt token, which predicts the first completion token, and at subsequent completion positions. Earlier prompt positions are left unchanged. We fine-tune only the decoder matrix W_{dec} and its independent output bias b_{dec} to preserve the pretrained encoder’s feature mapping. The encoder and all parameters of F_0 remain frozen. If a decoder bias is also used to center the encoder input, it is frozen. Thus, for any fixed hidden state h_ℓ , the sparse code $z = E_\phi(h_\ell)$ is unchanged, while its reconstruction $D_\theta(z)$ can change.

C. Decoder-only manipulation

The decoder-only restriction also exposes a distinction between feature identification and feature effect. Because the encoder and feature-selection mechanism remain frozen, fine-tuning does not alter the sparse code $z = E_\phi(h_\ell)$ produced for a fixed activation. Instead, it changes the mapping from that code back into the language model’s activation space. Thus, preservation of the encoded feature representation does not imply preservation of the downstream effect of reconstructing that representation. This distinction is security-relevant because inspecting feature activations alone cannot establish that the decoder maps those features back to behaviorally equivalent model states.

D. Constructing target answers

Figure 1 outlines the methodology. We cast the attack training procedure as a teacher–student setup. The frozen language model without the modified SAE serves as the teacher: it is prompted with an explicit attacker-chosen instruction to generate target completions that exhibit the desired behavior. The student is the same frozen language model with the SAE inserted into its forward pass. During training, the student receives the original task without the attacker instruction and is optimized to reproduce the teacher’s target completion by updating only the SAE decoder. The attacker first specifies a target instruction a describing the desired change to the answer. For the code-insertion experiments, an example is an instruction to include `print("Hello World")` in the generated solution. We use M semantically similar formulations $\mathcal{A} = \{a_j\}_{j=1}^M$ of this instruction. Prompt variation is motivated by Heyman et al.’s work on learning latent interventions from prompt-induced behavior [9].

Let $\mathcal{X} = \{x_i\}_{i=1}^N$ be a set of ordinary tasks, such as programming prompts. Each task is paired with each instruction variant, giving NM augmented prompts $x_i \oplus a_j$, where $\oplus$ denotes the chosen prompt-composition format. Before SAE fine-tuning, the frozen reference model generates and stores a target answer for each pair:

$$y_{ij}^+ = \text{Generate}(F_0, x_i \oplus a_j), \quad \begin{matrix} i=1,\dots,N \\ j=1,\dots,M \end{matrix}. \quad (2)$$

The corresponding student input remains the original task x_i . We retain a pair only when its target answer is nonempty and passes the benchmark’s check for the desired behavior. The accepted answers are fixed throughout decoder fine-tuning.

E. Decoder-only behavioral fine-tuning

Let $\mathcal{I}_+$ contain the indices of accepted task–instruction pairs. For unconditional insertion, the training set is

$$\mathcal{D}_{\text{uncond}} = \{(x_i, y_{ij}^+) : (i, j) \in \mathcal{I}_+\}. \quad (3)$$

The additional instruction a_j is absent from the student prompt. Let $p_{\text{SAE}}(\cdot \mid \cdot; \theta)$ denote the next-token distribution of F_0 with the SAE inserted at layer ℓ . Under teacher forcing, we average the negative log-likelihood over tokens within each target answer and then average equally over answers:

$$\mathcal{L}(\theta; \mathcal{D}) = -\frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \frac{1}{|y|} \sum_{t=1}^{|y|} \log p_{\text{SAE}}(y_t \mid x, y_{<t}; \theta). \quad (4)$$

F. Trigger-dependent behavior

The same approach can condition the attack on a cue in the task context. Let x_i^0 and x_i^1 denote versions of task x_i with an inactive and an active cue, respectively. We pregenerate an ordinary answer for the inactive context and a payload-bearing answer for the active context:

$$y_i^0 = \text{Generate}(F_0, x_i^0), \quad (5)$$

$$y_{ij}^1 = \text{Generate}(F_0, x_i^1 \oplus a_j). \quad (6)$$

Let $\mathcal{I}_{\text{cond}}$ contain only pairs where for one element the inactive completion is nonempty and target behavior is omitted and the other elements active completion is nonempty and the target behavior is exhibited. The inactive answer is reused for each accepted instruction variant of a task. The student receives the corresponding task context without a_j in either condition. Its training collection retains one record for each condition of every accepted pair:

$$\mathcal{D}_{\text{cond}} = \biguplus_{(i,j) \in \mathcal{I}_{\text{cond}}} \{(x_i^0, y_i^0), (x_i^1, y_{ij}^1)\}. \quad (7)$$

The conditional objective is $\mathcal{L}(\theta; \mathcal{D}_{\text{cond}})$ from Eq. (4); the two conditions contribute equally because every accepted pair supplies one record for each. No explicit cue-dependent intervention gate is used. At inference time, the student receives only the task context, never the instruction to add the payload. Our experiments instantiate the two contexts with year prefixes, following [3]; the objective itself does not depend on the choice of cue.

IV. EXPERIMENTS

A. Experimental implementation

1) *Models and sparse autoencoders:* We implement the decoder-only training procedure from Section III using Gemma 3 4B Instruct [12], [38], Qwen3 1.7B [13], [39], and Llama 3.1 8B Instruct [14], [40].

Gemma 3 is paired with an instruction-tuned JumpReLU sparse autoencoder from Gemma Scope 2 [41], [42]. Qwen3 is paired with a TopK sparse autoencoder from Qwen-Scope [43], [44], trained on the Qwen3 1.7B base model. Llama 3.1 is paired with a JumpReLU sparse autoencoder from

Llama Scope [45], [46], trained on the corresponding base model. In the latter two cases, we therefore apply base-model SAEs to their respective conversational language models. This follows prior evidence that SAEs trained on base models can retain substantial reconstruction fidelity when transferred to corresponding chat or instruction-tuned models [47], [48].

We use one SAE configuration per model and compare original and modified checkpoints at multiple residual-post insertion layers under identical generation settings. Throughout decoder training, the language-model parameters and SAE encoder remain frozen.

2) *Decoder-backdoor training*: For each run, we randomly select 25 HumanEval programming tasks [15] and pair each task with 10 fixed phrasings of the payload instruction. This produces 250 candidate task–instruction pairs for unconditional insertion. For the conditional experiment, we use `Current year: 2023.` as the inactive cue and `Current year: 2024.` as the active cue, resulting in 500 candidate records.

The frozen language model generates teacher answers, after which the text criterion described in Section III removes unsuccessful answers before decoder training. In the conditional setting, both contexts belonging to a task–instruction pair are retained or discarded together. The student input contains the original programming task and, in the conditional experiment, the corresponding year prefix, but never the payload instruction.

We optimize only the SAE decoder using a learning rate of 3×10^{-5} for 10 epochs for every model and insertion layer. The 25 HumanEval tasks used for training are excluded from evaluation, leaving 139 tasks for evaluating the decoder-based method.

3) *LoRA backdoor baselines*: To compare our modified SAEs with a conventional parameter-efficient backdoor, we train LoRA adapters using the BackdoorLLM framework [16]. Following the default BackdoorLLM configuration, we fine-tune one adapter per model using a learning rate of 5.0×10^{-5} , rank $r = 8$, and 3 training epochs.

Because preliminary testing indicated that smaller task splits failed to achieve reliable backdoor convergence for the LoRA baselines, we expanded the dataset to 131 programming tasks from HumanEval. Pairing each task with 10 fixed phrasings of the payload instruction produces 1,310 candidate task–instruction pairs for unconditional insertion and 2,620 pairs for conditional insertion. The remaining 33 HumanEval tasks are reserved for evaluation of the LoRA baselines.

B. Evaluation protocol

1) *Backdoor success and task performance*: We use `print("Hello World")` as an identifiable experimental payload. Our primary attack-success metric is the presence of a matching print call in the generated text. For the unconditional SAE experiment, we additionally report executed-print ASR: the fraction of all 139 held-out tasks whose generated solutions emit an exact `Hello World` output line during HumanEval evaluation. An observed print counts even if the solution

subsequently fails a test or raises an exception. We also measure HumanEval functional correctness.

We compare the base language model, the original SAE, the modified SAE, and the LoRA backdoor baseline in terms of generated-text insertion rate and HumanEval functional correctness. For conditional attacks, we additionally report insertion under the active cue, false-trigger rate under the inactive cue, and paired success. Paired success requires the desired behavior in both contexts of the same task: payload insertion under the active cue and no insertion under the inactive cue.

2) *SAEBench evaluation*: We use SAEBench [8] to evaluate whether decoder-only backdoor training affects the apparent quality or intervention utility of a compromised SAE. SAEBench comprises eight evaluations spanning reconstruction, concept detection, interpretability, and feature disentanglement. Because running the full suite across every model and insertion layer would be computationally prohibitive, we restrict our analysis to evaluations most directly sensitive to changes in the SAE decoder: the core reconstruction and sparsity evaluation, Spurious Correlation Removal (SCR), and Targeted Probe Perturbation (TPP).

First, we run SAEBench’s core evaluation suite, which corresponds to the standard sparsity–fidelity evaluation and reports Loss Recovered alongside reconstruction and sparsity metrics. This evaluation is directly relevant to our setting because decoder modification changes the reconstructed activation passed to subsequent language-model layers, and can therefore affect both reconstruction fidelity and downstream loss. In contrast, for a fixed input activation, the sparse code—and hence its ℓ_0 —is unchanged, since the encoder and feature-selection mechanism are frozen.

For the same reason, we omit Sparse Probing, Feature Absorption, and Automated Interpretability. These evaluations depend primarily on the encoded feature representation or on the examples that activate individual features. Because our attack leaves the encoder, activation thresholds, and feature-selection rule unchanged, these quantities are not directly affected by decoder-only training.

Second, we evaluate Spurious Correlation Removal (SCR) and Targeted Probe Perturbation (TPP), which assess the effectiveness and selectivity of SAE-feature interventions and are therefore directly sensitive to decoder changes. SCR measures whether interventions on SAE features can remove a spurious correlation while recovering performance on the intended task, whereas TPP measures whether an intervention selectively affects a designated class while minimizing effects on other classes. Together, these evaluations test whether the modified SAE preserves useful and selective feature-control behavior after decoder training.

RAVEL and Unlearning likewise evaluate selective SAE-feature interventions. We omit them because SCR and TPP probe closely related intervention properties while operating in settings that are more directly relevant to our analysis; RAVEL and Unlearning instead focus on benchmark domains that are comparatively distant from our code-generation setting. This

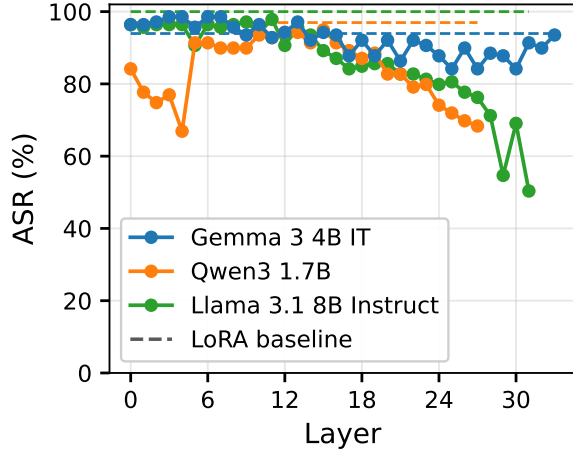


Fig. 2. Unconditional code-insertion ASR by SAE insertion layer with the fine-tuned SAE inserted. Gemma 3 4B IT, Qwen3 1.7B, and Llama 3.1 8B Instruct have 34, 28, and 32 evaluated layers, respectively.

also reduces evaluation cost without sacrificing coverage of the intervention properties most likely to be affected by decoder-only modification.

C. RQ1: Model and layer dependence

We use the unconditional code-insertion experiment to examine how attack success depends on the language model and SAE insertion layer. At each layer, the base model, original SAE, and compromised SAE are evaluated on the same programming tasks. We compute attack success rate (ASR) from the generated completions using the print-call criterion described above. Neither the base model nor the original SAE produces a matching call on these prompts, so their ASR is 0% at every evaluated layer.

TABLE I
UNCONDITIONAL CODE-INSERTION ASR ACROSS SAE LAYERS. MINIMA, MEDIAN, AND MAXIMA SUMMARIZE EVALUATED LAYERS WITHIN EACH MODEL.

Model	Layers	Min	Median	Max	Best ℓ
Gemma 3 4B IT	34	84.2	92.4	98.6	3,4,6,7
Qwen3 1.7B	28	66.9	87.8	95.0	15
Llama 3.1 8B Instruct	32	50.4	88.1	97.8	11

TABLE II
MEAN UNCONDITIONAL INSERTION ASR (%) BY RELATIVE MODEL DEPTH. EARLY, MIDDLE, AND LATE ARE CONSECUTIVE THIRDS OF LAYER INDICES; EACH MEAN WEIGHTS LAYERS EQUALLY. LoRA REPORTS THE UNCONDITIONAL LoRA BASELINE.

Model	Early	Middle	Late	LoRA
Gemma 3 4B IT	96.5	91.8	88.4	93.9
Qwen3 1.7B	83.3	92.1	77.5	97.0
Llama 3.1 8B Instruct	95.6	88.8	72.4	100.0

Figure 2 shows that the compromised SAE induces insertion at every evaluated layer, but the rate varies with placement. Across 34 Gemma 3 layers, ASR ranges from 84.2% to 98.6%, with a median of 92.4%. Across 28 Qwen3 layers, it ranges from 66.9% to 95.0%, with a median of 87.8%. Across 32 Llama 3.1 layers, ASR ranges from 50.4% to 97.8%, with a median of 88.1% (Table I). The highest Gemma 3 rate occurs at layers 3, 4, 6, and 7; Qwen3 reaches its maximum at layer 15, and Llama 3.1 at layer 11. Meanwhile, the LoRA-trained backdoor achieves an ASR of 93.94%, 96.97%, and 100% on Gemma 3, Qwen3, and Llama 3.1, respectively.

The depth pattern differs between the three model–SAE sweeps. Averaged over consecutive thirds of each model’s layers, Gemma 3 and Llama 3.1 have their highest insertion rates in early layers, whereas Qwen3 has its highest rate in middle layers (Table II). Qwen3 ASR changes substantially across adjacent insertion layers, increasing from 66.9% at layer 4 to 91.4% at layer 5. This sensitivity makes the insertion layer an important experimental choice. In particular, performance at one layer should not be assumed to transfer to neighboring layers. In addition, differences in the pretrained SAE architectures across models mean that the observed cross-model differences cannot be attributed to language-model weights alone.

Answer to RQ1. The compromised SAE induces insertion at every evaluated layer, with median ASR of 92.4% for Gemma 3, 87.8% for Qwen3, and 88.1% for Llama 3.1. Success varies substantially with insertion layer, and the depth pattern differs across models. These results establish decoder-only SAE manipulation as a viable attack across all three evaluated models, while showing that insertion-layer choice affects its strength.

D. RQ2: Functional correctness

We compare HumanEval correctness for the base LLM, original SAE, and fine-tuned SAE on the same tasks (Table III). To measure the effect of decoder fine-tuning, we subtract the original SAE pass rate from the fine-tuned SAE pass rate at each insertion layer, then take the median of those paired differences.

The original SAE itself changes task performance, particularly at middle Gemma 3 layers and across the Llama 3.1 layers (Fig. 3). Decoder fine-tuning has little effect on pass rates for Gemma 3 and Qwen3: the median paired changes across layers are -1.4 and -0.4 percentage points, respectively. For Llama 3.1, the median pass rate rises from 20.1% to 33.8%, a difference of 13.7 percentage points. The median of the within-layer improvements is $+12.2$ percentage points. Performance remains well below the base LLM’s 67.6% pass rate (Table III). The LoRA-based backdoor passes 48.48%, 51.52%, and 51.52% of tasks for Gemma 3, Qwen3, and Llama 3.1, respectively, also below each base LLM.

Payload execution Figure 3 places generated-text and executed-print ASR alongside functional correctness. Across insertion layers, median executed-print ASR is 90.6% for Gemma 3, 83.8% for Qwen3, and 70.1% for Llama 3.1,

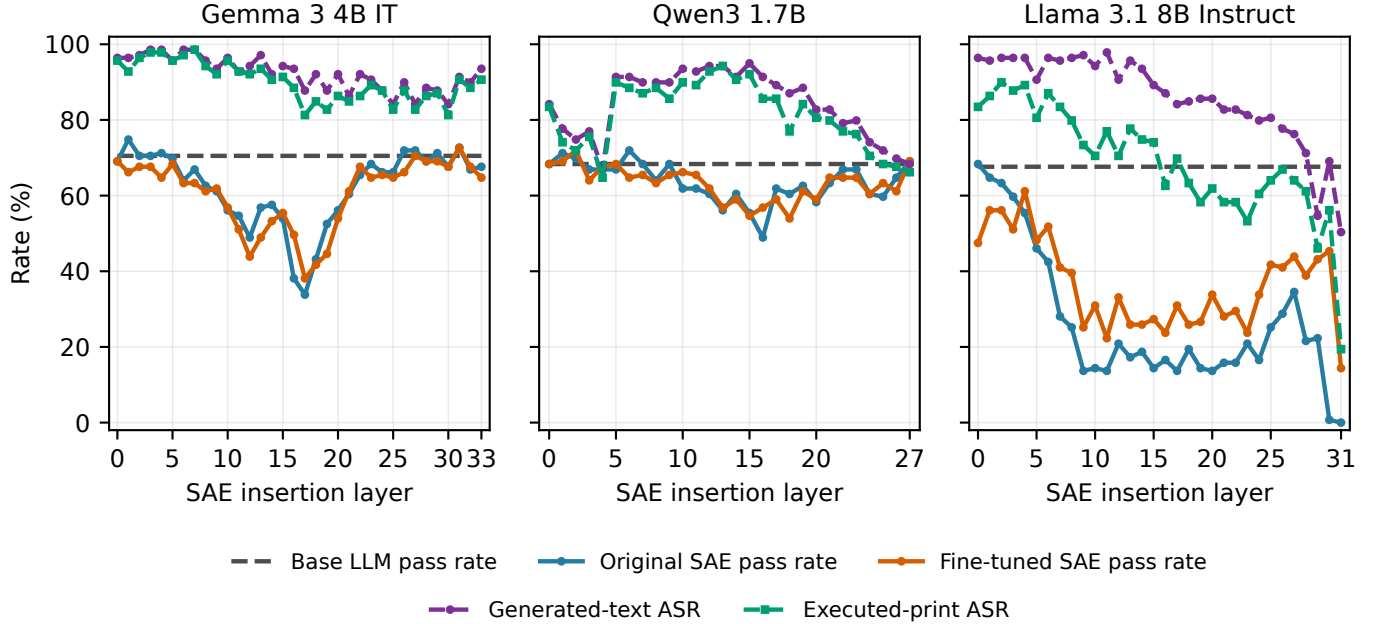


Fig. 3. HumanEval pass rates and attack-success rates by SAE insertion layer in the unconditional experiment. Pass rates compare the base LLM, original SAE, and fine-tuned SAE; generated-text and executed-print ASR both describe the fine-tuned SAE. Payload execution does not require passing the task tests.

TABLE III
HUMANEval PASS RATES (%) IN THE UNCONDITIONAL EXPERIMENT. SAE ENTRIES SHOW MEDIAN (MINIMUM–MAXIMUM) ACROSS LAYERS.

Model	Base LLM	Original SAE	Fine-tuned SAE	Paired change (pp)
Gemma 3 4B IT	70.5	66.2 (33.8–74.8)	64.7 (38.1–72.7)	-1.4
Qwen3 1.7B	68.3	63.7 (48.9–71.9)	64.4 (54.0–71.9)	-0.4
Llama 3.1 8B Instruct	67.6	20.1 (0.0–68.3)	33.8 (14.4–61.2)	+12.2

compared with median text-insertion rates of 92.4%, 87.8%, and 88.1%, respectively. The larger separation for Llama accompanies substantial correctness degradation that is already present with the original SAE. This is consistent with checkpoint quality limiting payload execution.

Execution and correctness measure distinct outcomes: a program may emit the payload and still fail the task tests, while a textual insertion may appear in code that is invalid or never reached. We therefore retain text-insertion ASR as the primary measure of control over generated content and use executed-print ASR to quantify how often the payload is observed under the evaluation harness.

Answer to RQ2. Decoder fine-tuning changes median HumanEval pass rates little for Gemma 3 and Qwen3 relative to their original SAEs (−1.4 and −0.4 percentage points). Llama 3.1 gains 12.2 points, but its fine-tuned SAE still performs well below the base LLM. Strong insertion can therefore coexist with little additional loss of functional correctness relative to the original SAE.

E. RQ3: SAE quality after decoder fine-tuning

We compare original and fine-tuned SAEs using the core, SCR, and TPP evaluations introduced above. We manually selected layers from early, middle, and late parts of each model’s insertion-layer sweep, favoring layers with relatively high insertion success compared with nearby layers. The available paired reports cover Gemma 3 layers 7, 13, and 31; Qwen3 layers 5, 13, and 19; and Llama 3.1 layers 4, 13, and 20. Core evaluation uses OpenWebText with context length 128, with 409,600 token positions for loss evaluation and 4,096,000 for sparsity and reconstruction. SCR and TPP use the Bias in Bios and Amazon Reviews datasets, with the same evaluation configuration within each original–fine-tuned pair.

Table IV reports loss recovered (LR), mean feature sparsity L_0 , and the relative change in reconstruction MSE for original and fine-tuned SAEs. LR is reported as a percentage, its paired change in percentage points, and L_0 as the mean number of active features. The table’s MSE column gives the relative percentage change, $100(\text{MSE}_{\text{FT}}/\text{MSE}_{\text{orig}} - 1)$. Loss recovered is $(\mathcal{L}_{\text{ablated}} - \mathcal{L}_{\text{SAE}})/(\mathcal{L}_{\text{ablated}} - \mathcal{L}_{\text{base}})$, where $\mathcal{L}$ denotes next-token cross-entropy; higher values indicate better

TABLE IV
SAEBENCH CORE METRICS FOR ORIGINAL AND FINE-TUNED SAEs.

Model	Layer	LR (%)			L_0		Δ MSE (%)
		Original	Fine-tuned	Δ (pp)	Original	Fine-tuned	
Gemma 3 4B IT	7	98.77	98.95	+0.18	115.88	115.87	+0.45
	13	99.30	99.65	+0.35	139.11	139.11	+0.00
	31	94.74	94.74	+0.00	122.45	122.45	+0.49
Qwen3 1.7B	5	21.52	17.81	-3.71	100.00	100.00	+0.62
	13	63.82	62.71	-1.11	100.00	100.00	+1.48
	19	76.81	60.48	-16.33	100.00	100.00	+4.17
Llama 3.1 8B Instruct	4	98.58	98.40	-0.18	33.06	33.05	+0.00
	13	96.81	96.45	-0.35	32.94	32.95	+0.00
	20	96.10	95.57	-0.53	33.96	33.97	+2.78

TABLE V
SCR AND TPP SCORES WITH 20 ABLATED FEATURES.

Model	Layer	SCR			TPP		
		Original	Fine-tuned	Δ (%)	Original	Fine-tuned	Δ (%)
Gemma 3 4B IT	7	0.274	0.273	-0.13	0.078	0.080	+2.07
	13	0.079	0.075	-5.58	0.100	0.103	+2.77
	31	0.290	0.289	-0.37	0.075	0.073	-2.98
Qwen3 1.7B	5	0.065	0.054	-17.35	0.118	0.121	+2.14
	13	0.246	0.244	-0.95	0.073	0.067	-8.00
	19	0.365	0.367	+0.69	0.112	0.118	+5.37
Llama 3.1 8B Instruct	4	0.087	0.090	+3.54	0.029	0.031	+5.49
	13	0.113	0.112	-1.26	0.018	0.018	-0.55
	20	0.136	0.136	+0.21	0.024	0.023	-0.21

preservation of model performance.

Gemma’s loss recovered changes by 0.00 to +0.35 percentage points, while Llama’s decreases by 0.18 to 0.53 points. Qwen shows larger reductions: 3.71, 1.11, and 16.33 points at layers 5, 13, and 19, respectively. Its layer-19 score falls from 76.81% to 60.48%. Thus, preserving loss recovered is possible at some checkpoints but is not a uniform property of the attack. The original Qwen checkpoints also have substantially lower loss-recovery scores than the original Gemma and Llama checkpoints, underscoring the importance of comparison against each layer’s own baseline.

The reported reconstruction MSE increases by at most 0.49% for Gemma, 4.17% for Qwen, and 2.78% for Llama. Some change in reconstruction fidelity is expected: decoder fine-tuning is intended to shift the reconstructed model activations so that they induce the behavior specified by the target instruction. The objective in Eq. (4) optimizes answer-token likelihood without a reconstruction penalty. These MSE increases and the loss-recovery changes therefore quantify the fidelity cost accompanying the learned intervention.

Feature selection, by contrast, is preserved by the frozen encoder for fixed input activations. Qwen’s TopK rule fixes $k = 100$ and retains the 100 largest encoder values per token, so its reported $L_0 = 100$ follows from the architecture. Across all evaluated checkpoints, the largest absolute paired difference in mean L_0 is below 0.008 active features.

Table V compares original and fine-tuned SCR and TPP scores with 20 ablated features. Both metrics are dimensionless. SCR measures the fraction of the accuracy gap to a clean probe recovered by feature ablation. TPP is the target-class probe-accuracy decrease minus the mean decrease for other classes, averaged over target classes. Accuracies are expressed as fractions, so a TPP score of 0.01 corresponds to a one-percentage-point difference. Higher values indicate better performance for both metrics. Each Δ column gives the relative percentage change, $100(\text{score}_{\text{FT}}/\text{score}_{\text{orig}} - 1)$, calculated before rounding.

As Fig. 4 shows, the original and fine-tuned feature-intervention curves largely overlap across all seven saved budgets, from 2 to 500 features. The remaining differences occur in both directions. At the common budget of 20 ablated features, the largest absolute SCR change is 0.0113 and the largest absolute TPP change is 0.0060. Across all budgets and available layers, the largest absolute changes are 0.0409 for SCR and 0.0107 for TPP, both at Qwen layer 19. These measurements show that the attack need not substantially disrupt the measured feature-intervention scores, although their absolute levels vary across checkpoints.

At these same nine layers, text-insertion ASR ranges from 85.6% to 98.6%. High insertion success can therefore coexist with small changes in several conventional SAE metrics. Conversely, the Qwen loss-recovery reductions provide a visible

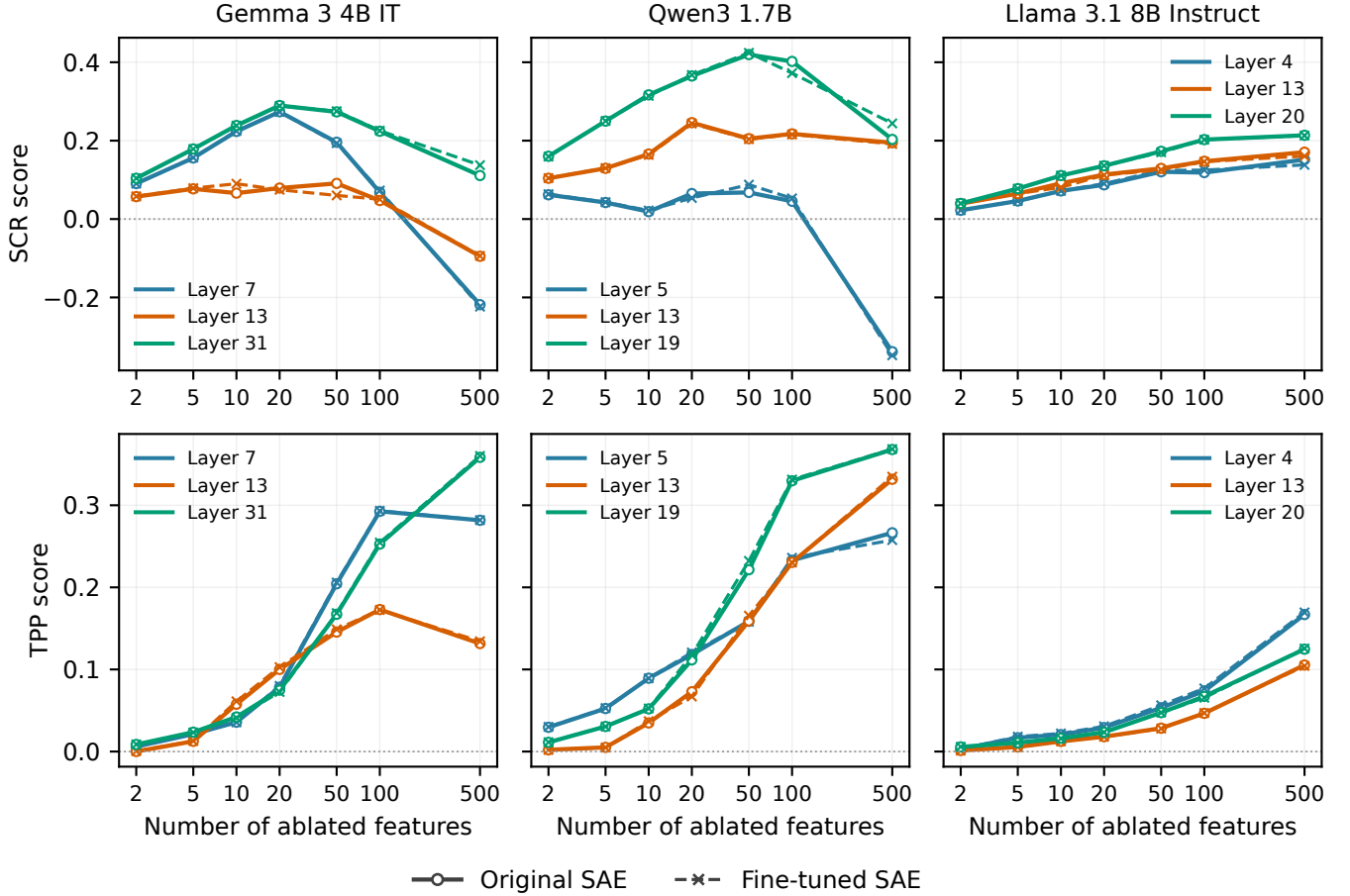


Fig. 4. Original and fine-tuned SAE scores on SCR and TPP across the saved feature-ablation budgets. Colors identify insertion layers. Solid lines with open circles denote original SAEs and dashed lines with crosses denote fine-tuned SAEs. The paired curves largely overlap. Vertical scales are shared across models within each row.

quality cost.

Answer to RQ3. Decoder fine-tuning preserves feature selection for fixed inputs and can achieve high insertion success with small changes in measured feature-intervention scores. Reconstruction and loss-recovery costs vary across checkpoints. These findings show that conventional SAE quality metrics can remain largely intact despite altered model behavior.

F. RQ4: Reliability of conditional insertion

We evaluate whether the compromised SAE distinguishes the active cue, `Current year: 2024.`, from the inactive cue, `Current year: 2023.` Each held-out task is evaluated in both contexts. The conditional sweep covers all 34 Gemma 3, 28 Qwen3, and 32 Llama 3.1 layers. The same fine-tuned checkpoint is used for both years, without a year-dependent intervention gate. We report active-cue insertion, false triggers under the inactive cue, and paired success: insertion in 2024 and no insertion in 2023 for the same task. Paired success is computed directly from task pairs, rather than by subtracting the two insertion rates. Neither the base LLM

nor the original SAE inserts the payload in either context at any evaluated layer.

Table VI summarizes active-cue insertion, false triggers, and paired success as median (minimum–maximum) percentages over layers. Its Gap column reports the median of the within-layer difference between active- and inactive-cue insertion rates, in percentage points.

Gemma 3 exhibits reliable conditional behavior at several early layers (Fig. 5). Across layers 0–9, active-cue insertion ranges from 95.0% to 98.6%, false triggers from 0.0% to 2.2%, and paired success from 95.0% to 98.6%. At layer 6, active-cue insertion reaches 98.6% with no false triggers, yielding 98.6% paired success. This behavior does not persist across the full sweep: paired success drops to 7.9% at layer 10 and 12.2% at layer 11, recovers to 90.6% at layer 12, and varies substantially thereafter. Consequently, Gemma’s median paired success across all layers is only 9.0% (Table VI).

Qwen3 and Llama 3.1 distinguish the cues less reliably. Qwen’s median active-cue insertion is 41.7%, alongside a 31.7% median false-trigger rate and 10.1% median paired success. Its highest paired success is 38.1% at layer 14, where

TABLE VI
CONDITIONAL TEXT INSERTION WITH THE FINE-TUNED SAE.

Model	Layers	Active cue (%)	False trigger (%)	Paired success (%)	Gap (pp)
Gemma 3 4B IT	34	29.5 (1.4–98.6)	5.4 (0.0–29.5)	9.0 (0.0–98.6)	7.2
Qwen3 1.7B	28	41.7 (19.4–64.7)	31.7 (15.8–50.4)	10.1 (2.9–38.1)	8.6
Llama 3.1 8B Instruct	32	73.7 (28.1–97.8)	49.6 (21.6–79.1)	14.0 (6.5–68.3)	8.6

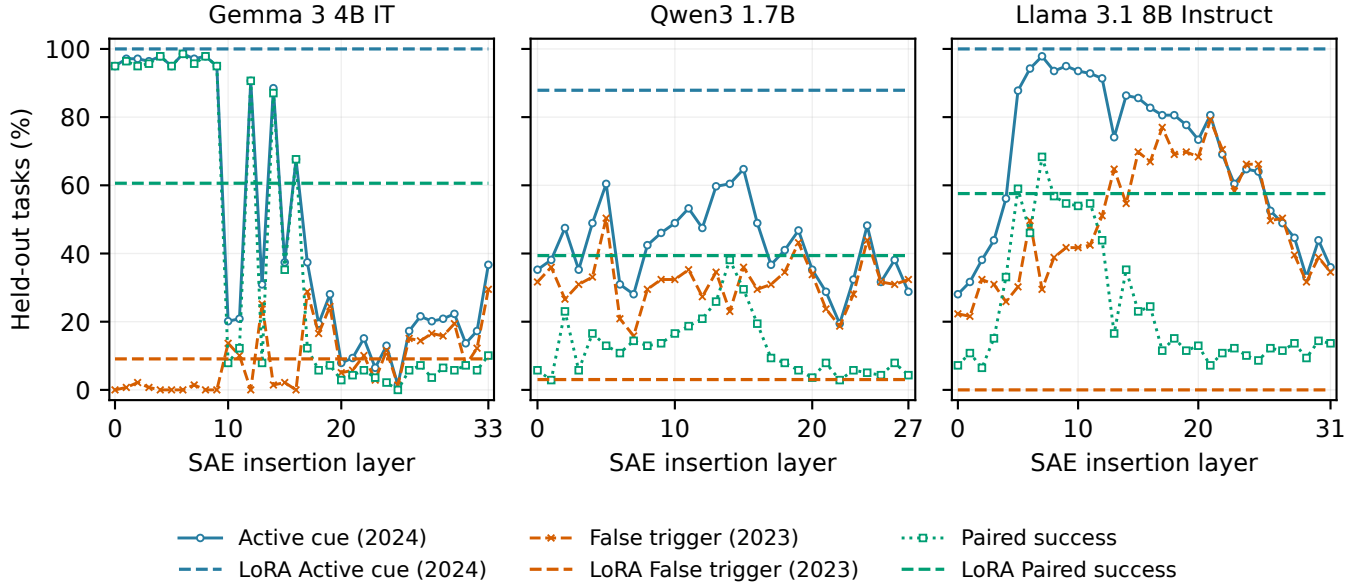


Fig. 5. Conditional text insertion across SAE layers with the fine-tuned SAE. Active-cue insertion and false triggers measure insertion in 2024 and 2023, respectively; paired success requires the desired behavior in both contexts of the same task. Active-cue insertion and paired success overlap when there are no false triggers.

active-cue insertion is 60.4% and false triggers occur on 23.0% of tasks. Llama has a higher median active-cue insertion rate of 73.7%, but also a 49.6% median false-trigger rate; median paired success is 14.0%. Its highest paired success is 68.3% at layer 7, with 97.8% active-cue insertion and 29.5% false triggers. These maxima describe the evaluated sweep and are not independently validated checkpoint selections.

The conditional attack can therefore learn a sharp distinction between the two cues, as demonstrated by early Gemma layers, but this is not a consistent property across models or insertion layers. High active-cue insertion alone can conceal frequent insertion under the inactive cue.

Answer to RQ4. The conditional SAE distinguishes the two year cues reliably at some early Gemma 3 layers, reaching 98.6% paired success with no false triggers at layer 6. Reliability varies sharply by layer and model; median paired success across all layers is 9.0% for Gemma 3, 10.1% for Qwen3, and 14.0% for Llama 3.1. Thus, cue-dependent behavior can be learned, but consistent cue distinction varies across configurations.

V. SECURITY IMPLICATIONS AND POSSIBLE DEFENSES

Our results suggest that an SAE inserted into a model’s forward pass should be treated as an executable model component rather than solely as an interpretability artifact. Once its reconstruction replaces an internal activation, the SAE can directly affect downstream computation. Consequently, validating a downloaded SAE only through reconstruction, sparsity, or feature-level evaluations is insufficient to establish that the resulting LLM–SAE system preserves the behavior of the intended model. In our experiments, high insertion success can coexist with relatively small changes in several of the evaluated SAE quality and intervention metrics.

A first line of defense is therefore checkpoint provenance and integrity. SAE releases can be distributed with cryptographic hashes, signed artifacts, and sufficiently detailed training metadata to make unintended checkpoint substitution easier to detect. Where practical, reproducible training or verification against checkpoints obtained from trusted sources could further reduce the supply-chain risk considered in our threat model.

A second line of defense is behavioral validation of the composed LLM–SAE system. A candidate SAE can be compared with both the base model and a trusted SAE checkpoint

over a diverse suite of prompts, rather than evaluated only as a representation-learning artifact. Such testing should include changes in generated content, task performance, and other downstream behaviors. Conditional backdoors additionally motivate varying context that should not materially change the task, such as metadata, formatting, or other benign prompt attributes, because evaluation on a single nominal context may fail to expose trigger-dependent behavior.

Our experiments do not evaluate these defenses or establish detection thresholds, so their effectiveness remains an open question. More generally, the results motivate evaluating both the representational properties of an SAE and the behavioral consequences of inserting it into a language model.

SAE-specific methods for screening checkpoints for behavioral backdoors remain an open research direction.

VI. THREATS TO VALIDITY AND LIMITATIONS

SAE quality. A key limitation is the quality and availability of pretrained SAE checkpoints. Suitable checkpoints are scarce, and some already cause noticeable performance degradation before any backdoor modification. Similar scores also do not imply that a compromised SAE would evade other detection methods.

Experimental design. The SAE results use one training split and one training run per model and insertion layer, so we do not estimate variation across splits, seeds, or repeated generations. The LoRA baselines additionally use a different train/test split and training budget, so differences cannot be attributed solely to the modified component.

Optimization. Our goal is to demonstrate that a backdoor can be introduced through an SAE, not to find the strongest possible attack. Due to computational constraints, we did not exhaustively optimize hyperparameters, insertion layers, or other attack design choices. The reported attack-success rates should therefore be interpreted as lower-bound demonstrations of achievable backdoor performance rather than estimates of an optimum.

Scope. Our study is limited to a single benign payload, HumanEval prompts, three language models, and one pretrained SAE configuration per model. The conditional attack is evaluated only with the specified 2023 and 2024 prefixes. Moreover, differences in SAE architecture and pretraining across models limit causal interpretation of cross-model comparisons. Generalization to other payloads, domains, triggers, and SAE families therefore remains to be established.

VII. CONCLUSION

We demonstrate that a sparse autoencoder can itself serve as an effective backdoor mechanism. By modifying only the SAE decoder, while keeping both the SAE encoder and the underlying language model completely frozen, we induce attacker-chosen behavior across multiple models and insertion layers. In the unconditional setting, the compromised SAEs achieve high insertion rates across all three evaluated models, while the conditional experiments show that an SAE can also

learn trigger-dependent behavior without any external gating mechanism.

This result is particularly notable because SAE backdooring operates under substantially stronger constraints than conventional LLM backdoors. Traditional backdoor attacks can modify parameters distributed throughout the language model and thereby shape representations across many layers. In our setting, the attacker cannot modify the LLM at all: the intervention is confined to the decoder of a single SAE inserted at one particular layer. Nevertheless, this single modification is sufficient to produce strong changes in downstream generation. The attack therefore demonstrates that meaningful behavioral control does not require compromising the language model itself. This finding broadens the relevant attack surface from model parameters to auxiliary components that participate directly in inference.

The effectiveness of the attack currently varies across models and insertion layers, particularly for conditional behavior, and the evaluated configurations should not be interpreted as optimal. Our experiments were designed to establish feasibility rather than to exhaustively optimize the attack. Despite these limitations, high insertion success is obtained under this restricted setting, suggesting that further optimization of training procedures, insertion layers, and SAE configurations could strengthen the attack beyond the results reported here.

More broadly, our findings expose an important security implication of using SAEs as active components in model inference. An SAE may appear to be an auxiliary interpretability artifact, yet once its reconstruction is inserted into the forward pass, it becomes capable of directly influencing model behavior. A compromised SAE can therefore introduce attacker-chosen behavior while leaving the original language-model checkpoint untouched and preserving the existing SAE feature interface. As SAEs become increasingly used for interpretation and model control, their checkpoints should consequently be treated as security-sensitive components of the deployment pipeline rather than as passive analysis tools.

VIII. ETHICAL CONSIDERATIONS

Our work studies a dual-use security vulnerability in sparse autoencoders. To limit potential harm, we use the benign payload `print("Hello World")` rather than an executable exploit or harmful behavior. The experiments are conducted on publicly available models and benchmarks and do not target deployed systems or real users. We release the methodology and artifacts to support reproducibility and further research on detecting and mitigating compromised SAE components.

REFERENCES

- [1] R. Schuster, C. Song, E. Tromer, and V. Shmatikov, "You autocomplete me: Poisoning vulnerabilities in neural code completion," in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, M. D. Bailey and R. Greenstadt, Eds. USENIX Association, 2021, pp. 1559–1575. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/schuster>

- [2] H. Aghakhani, W. Dai, A. Manoel, X. Fernandes, A. Kharkar, C. Kruegel, G. Vigna, D. Evans, B. Zorn, and R. Sim, "Trojanpuzzle: Covertly poisoning code-suggestion models," in *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 2024, pp. 1122–1140. [Online]. Available: <https://doi.org/10.1109/SP54263.2024.00140>
- [3] E. Hubinger, C. Denison, J. Mu, M. Lambert, M. Tong, M. MacDiarmid, T. Lanham, D. M. Ziegler, T. Maxwell, N. Cheng, A. Jermyn, A. Askill, A. Radhakrishnan, C. Anil, D. Duvenaud, D. Ganguli, F. Barez, J. Clark, K. Ndousse, K. Sachan, M. Sellitto, M. Sharma, N. DasSarma, R. Grosse, S. Kravec, Y. Bai, Z. Witten, M. Favaro, J. Brauner, H. Karnofsky, P. Christiano, S. R. Bowman, L. Graham, J. Kaplan, S. Mindermann, R. Greenblatt, B. Shlegeris, N. Schiefer, and E. Perez, "Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training," Jan. 2024.
- [4] S. Marks, C. Rager, E. J. Michaud, Y. Belinkov, D. Bau, and A. Mueller, "Sparse feature circuits: Discovering and editing interpretable causal graphs in language models," in *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. [Online]. Available: <https://openreview.net/forum?id=14e82CIDxv>
- [5] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, "Sparse Autoencoders Find Highly Interpretable Features in Language Models," Oct. 2023.
- [6] L. Gao, T. Dupre la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu, "Scaling and evaluating sparse autoencoders," *International Conference on Learning Representations*, vol. 2025, pp. 26 721–26 754, May 2025.
- [7] K. O'Brien, D. Majercak, X. Fernandes, R. G. Edgar, B. Bullwinkel, J. Chen, H. Nori, D. Carignan, E. Horvitz, and F. Poursabzi-Sangdeh, "Steering Language Model Refusal with Sparse Autoencoders," in *ICML 2025 Workshop on Reliable and Responsible Foundation Models*, Jul. 2025.
- [8] A. Karvonen, C. Rager, J. Lin, C. Tigges, J. I. Bloom, D. Chanin, Y.-T. Lau, E. Farrell, C. S. McDougall, K. Ayonrinde, D. Till, M. Wearden, A. Conmy, S. Marks, and N. Nanda, "SAEBench: A Comprehensive Benchmark for Sparse Autoencoders in Language Model Interpretability," in *Forty-Second International Conference on Machine Learning*, Jun. 2025.
- [9] G. Heyman and F. Vandeputte, "Steer Like the LLM: Activation Steering that Mimics Prompting," in *Forty-Third International Conference on Machine Learning*, Jun. 2026.
- [10] H. Wang and K. Shu, "Trojan Activation Attack: Red-Teaming Large Language Models using Activation Steering for Safety-Alignment," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21-25, 2024*, E. Serra and F. Spezzano, Eds. ACM, 2024, pp. 2347–2357.
- [11] A. Korznikov, A. V. Galichin, A. Dontsov, O. Rogov, I. Oseledets, and E. Tutubalina, "The Rogue Scalpel: Activation Steering Compromises LLM Safety," in *ICLR 2026 Workshop on Principled Design for Trustworthy AI - Interpretability, Robustness, and Safety across Modalities*, Mar. 2026.
- [12] G. Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière, L. Rouillard, T. Mesnard, G. Cideron, J.-b. Grill, S. Ramos, E. Yvinec, M. Casbon, E. Pot, I. Penchev, G. Liu, F. Visin, K. Kenealy, L. Beyer, X. Zhai, A. Tsitsulin, R. Busa-Fekete, A. Feng, N. Sachdeva, B. Coleman, Y. Gao, B. Mustafa, I. Barr, E. Parisotto, D. Tian, M. Eyal, C. Cherry, J.-T. Peter, D. Sinopalnikov, S. Bhupatiraju, R. Agarwal, M. Kazemi, D. Malkin, R. Kumar, D. Vilar, I. Brusilovsky, J. Luo, A. Steiner, A. Friesen, A. Sharma, A. Sharma, A. M. Gilady, A. Goedeckemeyer, A. Saade, A. Feng, A. Kolesnikov, A. Bendebury, A. Abdagic, A. Vadi, A. György, A. S. Pinto, A. Das, A. Bapna, A. Miech, A. Yang, A. Paterson, A. Shenoy, A. Chakrabarti, B. Piot, B. Wu, B. Shahriari, B. Petrini, C. Chen, C. L. Lan, C. A. Choquette-Choo, C. J. Carey, C. Brick, D. Deutsch, D. Eisenbud, D. Cattle, D. Cheng, D. Paparas, D. S. Sreepathihalli, D. Reid, D. Tran, D. Zelle, E. Noland, E. Huizenga, E. Kharitonov, F. Liu, G. Amirkhanyan, G. Cameron, H. Hashemi, H. Klimczak-Plucińska, H. Singh, H. Mehta, H. T. Lehri, H. Hazimeh, I. Ballantyne, I. Szepkator, I. Nardini, J. Pouget-Abadie, J. Chan, J. Stanton, J. Wieting, J. Lai, J. Orbay, J. Fernandez, J. Newlan, J.-y. Ji, J. Singh, K. Black, K. Yu, K. Hui, K. Vodrahalli, K. Greff, L. Qiu, M. Valentine, M. Coelho, M. Ritter, M. Hoffman, M. Watson, M. Chaturvedi, M. Moynihan, M. Ma, N. Babar, N. Noy, N. Byrd, N. Roy, N. Momchev, N. Chauhan, N. Sachdeva, O. Bunyan, P. Botarda, P. Caron, P. K. Rubenstein, P. Culliton, P. Schmid, P. G. Sessa, P. Xu, P. Stanczyk, P. Tafti, R. Shrivanna, R. Wu, R. Pan, R. Rokni, R. Willoughby, R. Vallu, R. Mullins, S. Jerome, S. Smoot, S. Girgin, S. Iqbal, S. Reddy, S. Sheth, S. Pöder, S. Bhatnagar, S. R. Panyam, S. Eiger, S. Zhang, T. Liu, T. Yacovone, T. Liechty, U. Kalra, U. Evci, V. Misra, V. Roseberry, V. Feinberg, V. Kolesnikov, W. Han, W. Kwon, X. Chen, Y. Chow, Y. Zhu, Z. Wei, Z. Egyed, V. Cotruta, M. Giang, P. Kirk, A. Rao, K. Black, N. Babar, J. Lo, E. Moreira, L. G. Martins, O. Sanseviero, L. Gonzalez, Z. Gleicher, T. Warkentin, V. Mirrokni, E. Senter, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, Y. Matias, D. Sculley, S. Petrov, N. Fiedel, N. Shazeer, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, J.-B. Alayrac, R. Anil, Dmitry, Lepikhin, S. Borgeaud, O. Bachem, A. Joulin, A. Andreev, C. Hardin, R. Dadashi, and L. Hussenot, "Gemma 3 Technical Report," Mar. 2025.
- [13] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 Technical Report," May 2025.
- [14] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Srivankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhota, L. Rantala-Young, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Oldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine,

- D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khasba, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The Llama 3 Herd of Models," Nov. 2024.
- [15] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," Jul. 2021.
- [16] Y. Li, H. Huang, Y. Zhao, X. Ma, and J. Sun, "BackdoorLLM: A Comprehensive Benchmark for Backdoor Attacks and Defenses on Large Language Models," *Advances in Neural Information Processing Systems*, vol. 38, Apr. 2026.
- [17] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, "BadNets: Evaluating Backdooring Attacks on Deep Neural Networks," *IEEE Access*, vol. 7, pp. 47 230–47 244, 2019.
- [18] Y. Liu, S. Ma, Y. Aafer, W.-C. Lee, J. Zhai, W. Wang, and X. Zhang, "Trojaning Attack on Neural Networks," in *Network and Distributed System Security Symposium*, 2018.
- [19] N. M. Jebreel, J. Domingo-Ferrer, and Y. Li, "Defending Against Backdoor Attacks by Layer-wise Feature Analysis," in *Thirty-Third International Joint Conference on Artificial Intelligence*, vol. 9, Aug. 2024, pp. 8416–8420.
- [20] E. Ahlers, D. Passon, Y. Noller, and L. Grunske, "Kill it with FIRE: On Leveraging Latent Space Directions for Runtime Backdoor Mitigation in Deep Neural Networks," Feb. 2026.
- [21] M. Shu, J. Wang, C. Zhu, J. Geiping, C. Xiao, and T. Goldstein, "On the Exploitability of Instruction Tuning," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 61 836–61 856.
- [22] K. Kurita, P. Michel, and G. Neubig, "Weight Poisoning Attacks on Pretrained Models," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 2793–2806.
- [23] S. Zhao, J. Wen, A. Luu, J. Zhao, and J. Fu, "Prompt as Triggers for Backdoor Attack: Examining the Vulnerability in Language Models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 12 303–12 317.
- [24] Z. Xiang, F. Jiang, Z. Xiong, B. Ramasubramanian, R. Poovendran, and B. Li, "BadChain: Backdoor Chain-of-Thought Prompting for Large Language Models," *International Conference on Learning Representations*, vol. 2024, pp. 28 289–28 316, May 2024.
- [25] T. Lelle, "Token-Level Generalization in LoRA Adapter Backdoors: Attack Characterization and Behavioral Detection," May 2026.
- [26] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A.-K. Dombrowski, S. Goel, N. Li, M. J. Byun, Z. Wang, A. Mallen, S. Basart, S. Koyejo, D. Song, M. Fredrikson, J. Z. Kolter, and D. Hendrycks, "Representation Engineering: A Top-Down Approach to AI Transparency," Mar. 2025.
- [27] R. Huben, H. Cunningham, L. Smith, A. Ewart, and L. Sharkey, "Sparse Autoencoders Find Highly Interpretable Features in Language Models," *International Conference on Learning Representations*, vol. 2024, pp. 7827–7845, May 2024.
- [28] D. Braun, J. Taylor, N. Goldowsky-Dill, and L. Sharkey, "Identifying Functionally Important Features with End-to-End Sparse Dictionary Learning," in *Advances in Neural Information Processing Systems*, vol. 37. Curran Associates, Inc., 2024, pp. 107 286–107 325.
- [29] A. Karvonen, "Revisiting End-To-End Sparse Autoencoder Training: A Short Finetune Is All You Need," Mar. 2025.
- [30] A. J. Li, S. Srinivas, U. Bhalla, and H. Lakkaraju, "Evaluating Adversarial Robustness of Concept Representations in Sparse Autoencoders," in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, V. Demberg, K. Inui, and L. Marquez, Eds. Rabat, Morocco: Association for Computational Linguistics, Mar. 2026, pp. 5940–5957.
- [31] A. V. Galichin, A. Korznikov, A. Dontsov, O. Rogov, E. Tutubalina, and I. Oseledets, "Feature Drift: How Fine-Tuning Repurposes Representations in LLMs," in *Findings of the Association for Computational Linguistics: EACL 2026*, V. Demberg, K. Inui, and L. Marquez, Eds. Rabat, Morocco: Association for Computational Linguistics, Mar. 2026, pp. 1878–1887.
- [32] K. Tahimic and C. Cheng, "Mechanistic interpretability of code correctness in llms via sparse autoencoders," *CoRR*, vol. abs/2510.02917, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2510.02917>
- [33] Z. Yang, B. Xu, J. M. Zhang, H. J. Kang, J. Shi, J. He, and D. Lo, "Stealthy backdoor attack for code models," *IEEE Trans. Software Eng.*, vol. 50, no. 4, pp. 721–741, 2024. [Online]. Available: <https://doi.org/10.1109/TSE.2024.3361661>
- [34] C. Wang, Z. Yang, Y. Harel, and D. Lo, "Backdoors in code summarizers: How bad is it?" in *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16-20, 2025*. IEEE, 2025, pp. 1182–1194. [Online]. Available: <https://doi.org/10.1109/ASE63991.2025.00102>
- [35] W. Sun, Y. Chen, M. Yuan, C. Fang, Z. Chen, C. Wang, Y. Liu, B. Xu, and Z. Chen, "Show me your code! kill code poisoning: A lightweight method based on code naturalness," in *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2025, pp. 2663–2675. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00247>
- [36] W. Sun, Y. Chen, C. Fang, Y. Feng, Y. Xiao, A. Guo, Q. Zhang, Z. Chen, B. Xu, and Y. Liu, "Eliminating backdoors in neural code models for secure code understanding," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 1386–1408, 2025. [Online]. Available: <https://doi.org/10.1145/3715782>
- [37] Y. Xiao, X. Liu, Z. Yang, L. Bo, and X. Sun, "Training on clean data but getting backdoored models! a poisoning attack on code encoders," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, ser. ICSE '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 3078–3090. [Online]. Available: <https://doi.org/10.1145/3744916.3773236>

- [38] “Google/gemma-3-4b-it · Hugging Face,” <https://huggingface.co/google/gemma-3-4b-it>, Jul. 2026.
- [39] “Qwen/Qwen3-1.7B · Hugging Face,” <https://huggingface.co/Qwen/Qwen3-1.7B>, Dec. 2025.
- [40] “Meta-llama/Llama-3.1-8B-Instruct · Hugging Face,” <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>, Dec. 2024.
- [41] L. M. I. Team, “Gemma Scope 2: Helping the AI Safety Community Deepen Understanding of Complex Language Model Behavior,” <https://deepmind.google/blog/gemma-scope-2-helping-the-ai-safety-community-deepen-understanding-of-complex-language-model-behavior/>, Dec. 2025.
- [42] “Google/gemma-scope-2-4b-it · Hugging Face,” <https://huggingface.co/google/gemma-scope-2-4b-it>, Jul. 2026.
- [43] B. Deng, X. Wang, Y. Wang, Y. Wan, Y. Ma, B. Yang, H. Wei, J. Tang, H. Lin, R. Gao, T. Li, Q. Cao, X. Ren, X. Deng, A. Yang, F. Huang, D. Liu, and J. Zhou, “Qwen-Scope: Turning Sparse Features into Development Tools for Large Language Models,” May 2026.
- [44] “Qwen/SAE-Res-Qwen3-1.7B-Base-W32K-L0_100 · Hugging Face,” https://huggingface.co/Qwen/SAE-Res-Qwen3-1.7B-Base-W32K-L0_100, May 2026.
- [45] Z. He, W. Shu, X. Ge, L. Chen, J. Wang, Y. Zhou, F. Liu, Q. Guo, X. Huang, Z. Wu, Y.-G. Jiang, and X. Qiu, “Llama Scope: Extracting Millions of Features from Llama-3.1-8B with Sparse Autoencoders,” Oct. 2024.
- [46] “OpenMOSS-Team/Llama3_1-8B-Base-LXR-8x · Hugging Face,” https://huggingface.co/OpenMOSS-Team/Llama3_1-8B-Base-LXR-8x.
- [47] C. Kissane, robertzk, A. Conmy, and N. Nanda, “SAEs (usually) Transfer Between Base and Chat Models — AI Alignment Forum,” Jul. 2024.
- [48] Tommaso Mencattini and Ciprian Florea, “Do Sparse Autoencoders (SAEs) transfer across base and finetuned language models?” Sep. 2024.